

배치 개발자 가이드

처음 CPF Batch 를 사용하는 개발자가 Job 을 만들고 재실행까지 검증하는 실무 매뉴얼

공식 문서 03

기준: 최신 로컬 Source · 처음 사용하는 사용자 관점

전체 목차

필요한 항목을 누르면 해당 장으로 바로 이동합니다.

- 1 [배치 개발 Quick Finder](#)
- 2 [자주 쓰는 명령·실행 구조](#)
- 3 [첫 Batch Job 만들기](#)
- 4 [Job·Step 과 Annotation](#)
- 5 [Tasklet 과 Chunk 선택](#)
- 6 [Reader·Processor·Writer](#)
- 7 [Parameter·Instance·Execution](#)
- 8 [Transaction·Checkpoint·Restart](#)
- 9 [Retry·Skip·Reprocess](#)
- 10 [Partition·Worker](#)
- 11 [Scheduler·Misfire](#)
- 12 [Center-Cut·Preview](#)
- 13 [외부 연계·UNKNOWN_RESULT](#)
- 14 [Test·ADM 확인](#)
- 15 [운영 인계·Reference](#)

1. 배치 개발 Quick Finder

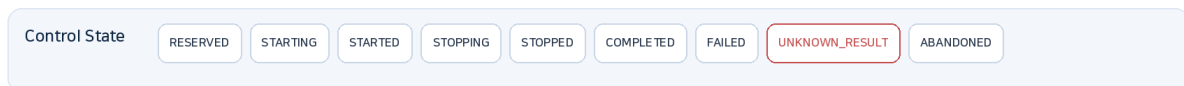
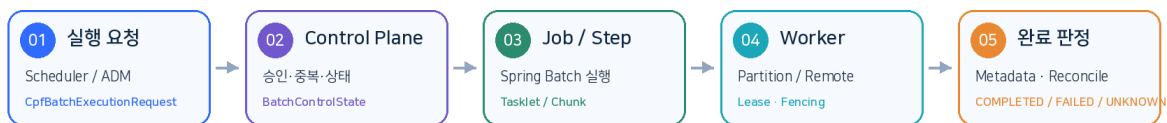
[전체 목차로](#)

Job 을 만들기 전에 “한 번 실패했을 때 어디부터 다시 시작해야 하는가”부터 결정합니다. 아래 표에서 업무 형태를 먼저 고릅니다.

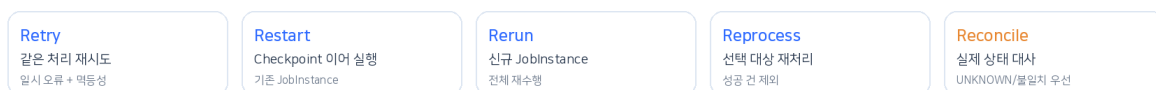
하려는 일	먼저 선택	상세
한 번 실행되는 단순 작업	Tasklet	Tasklet/Chunk
많은 행 반복 처리	Chunk	Reader/Writer
실패 후 이어서 실행	Restart + Checkpoint	Restart
일시 오류만 다시 시도	Retry	Retry/Reprocess
일부 대상만 다시 처리	Reprocess	Retry/Reprocess
대량 병렬 처리	Partition + Worker	Partition
정해진 시간 실행	Scheduler	Scheduler
대상 선정·분할·대사	Center-Cut	Center-Cut
외부 처리 여부 불명	UNKNOWN + Reconcile	UNKNOWN_RESULT LT

CPF Batch 실행 생명주기

실행 요청, Control Plane, Spring Batch 실행, Worker 처리와 복구 판단을 하나의 흐름으로 연결합니다.



실패 이후 선택



Batch 는 등록 → 실행 → Step 처리 → 완료/중지/실패 → Restart/Reprocess/Reconcile 의 Lifecycle 로 봅니다.

2. 자주 쓰는 명령·실행 구조

[전체 목차로](#)

Batch 개발도 같은 `cpf-dev.ps1`을 사용합니다. Runtime 명령과 Job 운영 명령을 구분해 기억하면 됩니다.

```
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 [Action] -ResourceProfile local
```

2.1 개발·실행 명령

용도	명령어	세부 용도	주요 옵션	참고사항
Batch 실행	run-batch	Batch Runtime 만 별도 기동	-ResourceProfile	Batch Job 개발 시 사용
실행 상태	status	Local Runtime 상태 확인	-	Worker/Runtime 확인
실행 종료	stop	Local Runtime 종료	-	개발 종료 시 사용

2.2 빌드·검증 명령

용도	명령어	세부 용도	주요 옵션	참고사항
전체 빌드	build	Batch 포함 전체 Compile/Gate	-ResourceProfile	코드 변경 후 기본 확인
Java Test	test	Job/Step Test 포함 Java Test	-ResourceProfile	Test 만 빠르게 볼 때
빠른 검증	verify-fast	계약·Catalog·정적 Gate	-ResourceProfile	평소 기본 검증
전체 검증	verify-full	DB/Runtime 까지 가능한 범위	-ResourceProfile / -OutputRoot	마무리 검증

Job 실행·Restart·Reprocess 개별 Job 실행과 운영 조치는 `cpf-dev.ps1`의 개발 Shell 명령이 아니라 ADM/Batch 운영 API의 영역입니다. 개발자는 Job 코드를 만들고 Runtime/Test를 검증하고, 운영 조작은 05 배치 운영 가이드의 절차를 따릅니다.

2.3 실행 구조

구성	역할	개발자가 신경 쓸 것
Scheduler	실행 시점/요청	Job 코드와 Schedule 정책을 분리
Control Plane	정의/실행/상태 제어	운영 API 를 업무 Job 에서 직접 구현하지 않음
Worker	Job/Step 실제 실행	재시작·동시 실행·외부 Side Effect
Agent	승인된 Runtime 작업	Job 업무 로직과 분리
Center-Cut	대량 대상·진행·대사	Preview/분할/Reconcile

3. 첫 Batch Job 만들기

[전체 목차로](#)

1. Job ID 와 입력 Parameter 를 정합니다.
2. Tasklet 또는 Chunk 를 선택합니다.
3. 실패 시 Restart 할 지점을 정합니다.
4. 중복 실행/동시 실행 정책을 정합니다.
5. 외부 Side Effect 가 있으면 Idempotency/UNKNOWN 처리 방법을 정합니다.
6. 정상 → 의도적 실패 → Restart Test 를 한 번에 작성합니다.

기본 셋업은 이미 되어 있다고 가정 이 문서는 Spring Batch 설치나 Project 생성보다 업무 Job 구현에 집중합니다. Runtime/DB 가 준비되지 않은 경우에만 07 Specification 의 환경 Reference 를 확인합니다.

4. Job-Step 과 Annotation

[전체 목차로](#)

현재 Source 에는 Batch Job 을 연결하는 두 계약이 있습니다. ****일반 Spring Batch Job Bean****과 ****CPF Annotation Step Handler****를 섞지 말고, 프로젝트에서 채택한 실행 모델에 맞는 쪽만 사용합니다.

용도	사용 계약	개발자가 판단할 것
Spring Batch Job Bean	com.cpf.batch.api.CpfBatchJob(id, name, ownerDomain)	Education/일반 JobBuilder 기반 Job 처럼 Spring Batch Job Bean 을 등록할 때
Annotation Step Handler	com.cpf.batch.api.annotation.CpfBatchJob + @CpfBatchStep	BatchStepHandler Runtime 으로 업무 Step method 를 직접 연결할 때

일반 JobBuilder 기반 업무 Job 은 현재 Education Sample 과 같이 Job Bean 에 `@CpfBatchJob(id, name, ownerDomain)` 을 붙이는 패턴을 먼저 참고합니다.

```
@Bean
@com.cpf.batch.api.CpfBatchJob(
    id = "BEDUAA0001", name = "EDUChunk 교육배치", ownerDomain = "EDU")
public Job memberDailyJob(JobRepository jobRepository, Step memberDailyStep) {
    return new JobBuilder("MBR_DAILY_MEMBER", jobRepository)
        .start(memberDailyStep)
        .build();
}
```

반대로 Annotation Step Handler 모델을 선택한 경우에만 아래 class/method Annotation 을 사용합니다. Runtime 은 이 계약을 `CpfAnnotatedBatchStepHandler` 로 소비합니다.

Annotation	주요 속성	의미
@CpfBatchJob (annotation package)	value / restartable / maxConcurrentExecutions	Handler 형 Job ID·재시작·동시 실행 정책
@CpfBatchStep	value / order / idempotent	Handler 형 Step ID·순서·재실행 안전성

5. Tasklet 과 Chunk 선택

[전체 목차로](#)

선택	잘 맞는 일	고려
Tasklet	파일 이동, 단일 SQL, 관리 명령	작업 전체가 실패 단위가 되기 쉬움
Chunk	행 단위 조회→가공→저장	chunk size 가 commit/restart 단위
Partition	독립 범위로 나눌 수 있는 대량 처리	partition key 와 worker 소유권

단순히 “건수가 많다”는 이유만으로 Partition 부터 선택하지 않습니다. Chunk 로 안정적인 재시작 경계를 만든 뒤 병렬화가 필요한지 판단하는 편이 안전합니다.

6. Reader·Processor·Writer

[전체 목차로](#)

Chunk Job 의 Reader/Processor/Writer 는 현재 CPF Education Sample 에서도 Spring Batch Item 계약과 `StepBuilder` 를 그대로 사용합니다. CPF 는 실행 Context·운영 추적·Worker/Control 경계를 연결하고, 업무 데이터 읽기/가공/쓰기는 표준 Batch 계약으로 구현합니다.

구성	업무 역할	Restart 관점
ItemReader	처리 대상을 안정된 순서로 읽음	마지막 읽은 위치를 재현할 수 있어야 함
ItemProcessor	검증·변환	가능하면 외부 Side Effect 를 넣지 않음
ItemWriter	DB/파일 등 최종 반영	Commit 경계와 멍등성 확인

```

.<MemberRow, MemberResult>chunk(500)
.transactionManager(transactionManager)
.reader(reader)
.processor(processor)
.writer(writer)
.build();

```

Processor 의 외부 호출 Processor 에서 외부 시스템을 변경하면 Chunk rollback 과 외부 반영이 어긋날 수 있습니다. 꼭 필요하면 Idempotency Key 와 결과불명 처리까지 같이 설계합니다.

7. Parameter·Instance·Execution

[전체 목차로](#)

개념	쉽게 말하면	왜 중요한가
Job Parameter	이번 회차를 구분하는 입력	같은 회차 Restart 인지 새 실행인지 결정
JobInstance	논리적인 실행 회차	식별 Parameter 가 같으면 동일 Instance 가 될 수 있음
JobExecution	실제 한 번의 실행 시도	실패/중지/Restart 이력이 쌓임
StepExecution	Step 별 실행 시도	어디서 몇 건 처리했는지 확인
ExecutionContext	재시작 정보	Reader/Partition 의 진행 위치 저장

예를 들어 영업일 배치라면 `businessDate`를 식별 Parameter로 돌지, 재처리 회차를 별도 Parameter로 돌지 업무 규칙을 먼저 정합니다.

8. Transaction·Checkpoint·Restart

[전체 목차로](#)

Chunk size는 성능 숫자이면서 동시에 실패/Commit 단위입니다. 1,000 건마다 Commit하면 중간 실패 시 그 Chunk의 미 Commit 범위가 다시 처리될 수 있습니다.

상황	설계 질문	확인 Test
Chunk 실패	어디까지 rollback되는가?	N번째 Writer 실패 후 DB 건수
Restart	어디부터 다시 읽는가?	ExecutionContext/Reader 위치
완료 Step	Restart 때 다시 도는가?	StepExecution 이력
중복	이미 외부 반영된 건은?	Idempotency/Reconcile

Stop과 Kill은 다름 Stop은 정상적인 중지 요청으로 Checkpoint/상태 전이를 기대할 수 있지만 Process Kill은 그렇지 않습니다. Restart 가능성을 Kill 상황에서도 Test해야 합니다.

9. Retry·Skip·Reprocess

[전체 목차로](#)

방식	언제	하지 말아야 할 때
Retry	일시적인 동일 작업 재시도	결과가 UNKNOWN 인 Side Effect
Skip	일부 오류 레코드를 건너뛰어도 업무적으로 허용	데이터 누락이 허용되지 않는 업무
Restart	실패 Execution 을 Checkpoint 부터 재개	새 업무 회차를 만들 때
Reprocess	특정 실패 업무 건만 다시 처리	원인을 해결하지 않은 전체 재실행
Abandon	운영 판단으로 해당 Execution 을 재시작 대상에서 제외	원인 확인 전 단순 실패 정리 용도

`Abandon`은 개발 코드의 재시도 API 가 아니라 운영 제어입니다. Source 의 `BatchExecutionControlPort.abandon(...)` 과 `ABANDONING/ABANDONED` 상태로 관리되며, 실제 조작 절차는 05 배치 운영 가이드를 따릅니다.

10. Partition·Worker

[전체 목차로](#)

Partition 은 “병렬 Thread 수”가 아니라 독립적으로 소유·재시작 가능한 처리 범위를 정의하는 것입니다.

- Partition key 범위가 겹치지 않게 설계합니다.
- Worker 가 죽은 뒤 Lease/Claim 을 다른 Worker 가 가져갈 수 있어야 합니다.
- 오래된 Worker 가 늦게 쓰는 것을 Fencing/Version 으로 차단합니다.
- maxConcurrentExecutions 와 Job 자체 동시 실행 정책을 함께 확인합니다.

11. Scheduler·Misfire

[전체 목차로](#)

Schedule 은 Job 로직과 분리합니다. 실행 시각을 놓쳤을 때 어떻게 할지(Misfire)를 운영과 합의해야 합니다.

정책 예	의미	적합한 업무
즉시 1 회	놓친 회차를 지금 1 번 실행	최신 1 회만 필요
다음 회차	놓친 실행을 버림	과거 회차 의미 없음
누락 회차 처리	놓친 회차를 순서대로 처리	회차별 결과가 모두 필요

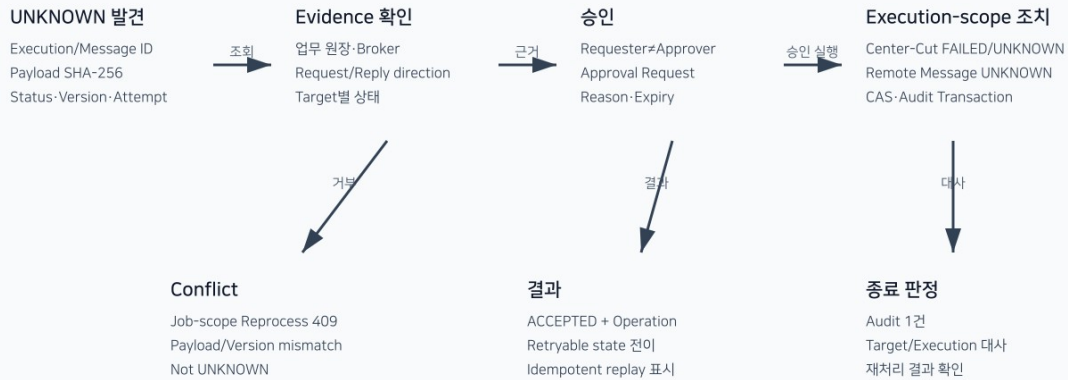
12. Center-Cut·Preview

[전체 목차로](#)

Center-Cut 은 단순 대량 Job 이름이 아니라 대상 선정 → Preview → 분할 → 실행 → 진행 → 실패 대상 → Reconcile 흐름을 운영과 함께 닫는 방식입니다.

Batch 결과 불명·부분 실패 Reconciliation

Center-Cut과 Remote Message는 실행·메시지 단위의 승인된 조치만 허용한다.



입력·처리·DB 반영·오류 건수를 대사해 실제 완료 여부를 판단합니다.

단계	개발자가 보장할 것
Preview	실제 실행과 같은 대상 조건으로 건수/범위를 계산
분할	중복 없는 target/partition key
실행	실행 식별자와 진행 수치
실패	실패 대상 재식별 가능
대사	입력/성공/실패/반영 건수가 설명 가능

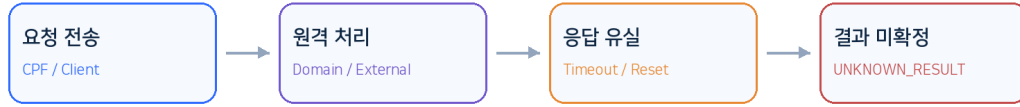
13. 외부 연계·UNKNOWN_RESULT

[전체 목차로](#)

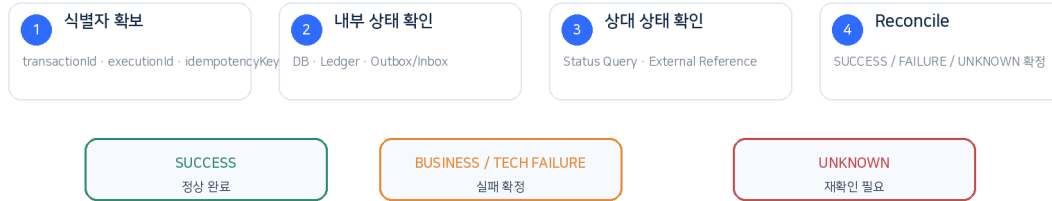
DB Commit 후 외부 시스템 응답을 받지 못했다면 “실패”라고 단정하면 중복 처리 위험이 있습니다. CPF 는 이런 경계를 UNKNOWN_RESULT/Reconcile 대상으로 분리합니다.

UNKNOWN_RESULT 판단 흐름

Timeout이나 응답 유실은 곧바로 실패를 뜻하지 않습니다. 실제 처리 여부를 먼저 확정합니다.



확인 순서



금지: 결과가 미확정인 상태에서 새 Idempotency Key로 동일 Side Effect 요청을 즉시 재실행하지 않습니다.

응답 유실/Timeout으로 상대 처리 여부가 확정되지 않으면 별도 확인 경로가 필요합니다.

- 상대 조회에 사용할 업무 Key/Idempotency Key를 남깁니다.
- UNKNOWN 상태에서는 무조건 자동 Retry 하지 않습니다.
- Reconcile 후 처리됨/미처리/판정불가를 구분합니다.

14. Test·ADM 확인

[전체 목차로](#)

Test	꼭 확인
정상	예상 처리 건수와 Commit 건수
중간 실패	Restart 위치와 중복 여부
중복 실행	같은 Parameter/Job 동시 실행 차단
Worker Kill	Claim/Lease 회수와 재개
외부 Timeout	UNKNOWN 분류와 Reconcile
Partition 일부 실패	성공/실패 Partition을 구분해 재실행

```

pwsh .\cpf-tools\build\tools\cpf-dev.ps1 run-batch
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 verify-fast
  
```

ADM에서는 Batch / Center-Cut, Executions, Runtime Topology, Worker Pools, Scheduler HA, Recovery / Unknown 화면을 이용해 개발 중에도 실행 상태가 실제로 보이는지 확인합니다.

15. 운영 인계·Reference

[전체 목차로](#)

배치 개발 완료 시 운영자에게 코드 설명보다 “언제 실행하는가, 어떤 Parameter 가 필요한가, 실패하면 무엇을 해도 되는가”를 전달해야 합니다.

인계 항목	최소 내용
실행 조건	Schedule/수동 실행 가능 여부
Parameter	필수값·식별값·예시
Restart	가능 상태와 시작 지점
Reprocess	대상 선정 Key 와 승인 필요 여부
Abandon	재시작 불가 처리 조건과 승인/사유
동시 실행	허용 수·중복 방지 기준
관측	Job/Execution/Transaction ID 와 ADM 위치

15.1 Job Pack·Artifact 를 언제 신경 쓰는가

일반 업무 Job 작성 중에는 Artifact 설치 내부를 알 필요가 없습니다. 다만 Job 을 Worker/Control Plane 에 등록·배포하는 인계 단계에서는 ****Job Pack 계약****을 알아야 합니다.

계약	용도	개발자가 확인할 것
BusinessJobProvider	Domain Job 을 Runtime 에 제공	manifest()와 resolveJob(jobId) 연결
JobPackManifest	Job Pack ID/Owner/Artifact/Version/Job 목록 계약	jobId·restartable·parameter·execution policy 가 인계 내용과 일치
ArtifactManifest	배포 Artifact 의 hash/signature/version 계약	업무 Job 코드에서 직접 조작하지 않고 배포/Agent 경계에서 확인

Job Pack 등록/배포를 담당하지 않는 업무 개발자는 위 세 계약의 존재와 인계 항목만 알면 충분합니다. 상세 필드와 운영 적용은 07 Specification 및 운영 문서를 사용합니다.

상세 계약 정확한 Batch Public API, State, Operation, Config 는 07 Specification 에서 확인합니다. 이 문서는 Job 구현·재실행 판단에 필요한 실무 흐름을 우선합니다.